



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

AI for Retail

Course Code: C398

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	17 August 2026	Initial release. Two topics, nine hands-on labs, one connected Bloom & Brew retail scenario.	Dr. Alfred Ang

TABLE OF CONTENTS

Introduction.....	4
Course Learning Outcomes.....	4
Before You Start — Preparation.....	4
Topic 01 — Introduction to AI for Retail.....	5
Lab 1 — Brief an AI Assistant Like a Retail Manager.....	5
Lab 2 — Generate a Product Catalogue with AI.....	7
Lab 3 — Campaign Copy and AI Product Visuals.....	9
Lab 4 — Customer Data Privacy and Responsible AI.....	12
Topic 02 — Applying AI to Retail Operations and Customer Experience.....	14
Lab 5 — Build a Grounded Store Service Assistant.....	15
Lab 6 — Personalised Recommendations and Targeted Promotions.....	17
Lab 7 — Demand Forecasting and Inventory Planning with AI.....	19
Lab 8 — AI for Pricing, Merchandising and Sales Analytics.....	21
Lab 9 — Build a Simple AI Workflow for a Daily Retail Task.....	24
Wrap-Up - Putting AI to Work in Your Store.....	26
Next Steps.....	27
Glossary.....	27

Introduction

This Learner Guide accompanies the course AI for Retail (C398), conducted by Tertiary Infotech Academy Pte Ltd. It provides step-by-step instructions for all nine hands-on labs, organised into the two topics that follow the course slides and Lesson Plan. No programming experience is required - every lab is done in a browser with an AI assistant and a spreadsheet.

All nine labs run on one fictional retailer, Bloom & Brew - a specialty home, gift and coffee retailer with six outlets and an online store. The synthetic data files in labs/resources/ carry through the whole day, so each lab builds on the one before it. Work through the labs in order; if you fall behind, each lab states the file it starts from so you can rejoin.

Course Learning Outcomes

- LO1: Describe the AI landscape from generative AI to AI agents, and identify high-value AI use cases across the retail value chain.
- LO2: Use AI assistants such as ChatGPT, Claude, Microsoft Copilot and Google Gemini to complete everyday retail tasks with well-structured prompts.
- LO3: Generate on-brand product descriptions, marketing copy and product visuals with AI, and check them before they reach a customer.
- LO4: Apply customer data privacy, governance and responsible-AI practices when using AI tools on retail data.
- LO5: Deploy AI for customer service, personalised recommendations and targeted promotions in a retail setting.
- LO6: Apply AI to demand forecasting, inventory planning, pricing, merchandising and sales analytics, and build simple AI workflows for daily retail tasks.

Before You Start — Preparation

What you need

- A laptop with a modern browser (Chrome or Edge) and a stable internet connection.
- A free ChatGPT account (chatgpt.com) - the assistant used in most labs.
- A free Claude account (claude.ai) and access to Microsoft Copilot (copilot.microsoft.com) or Google Gemini (gemini.google.com) for the comparison lab.
- A Google account for Google Sheets, used in the analytics and workflow labs.
- The synthetic Bloom & Brew data from the course repository: [labs/resources/](#) (product catalogue, transactions, customers, sales history, feedback, store policies, FAQ and brand voice guide).
- No programming experience and no software installation is required.

Verify your setup

Before you start, open each AI assistant you plan to use and send the message 'Reply with OK if you can read this.' If each one replies, you are ready. Then download the labs/resources/ folder and open products.csv in a spreadsheet to confirm the files opened correctly.

Conventions used in every lab

- Text shown in a PROMPT block is meant to be copied and pasted into the AI assistant named in that step.
- Placeholders such as <YOUR STORE NAME> or <YOUR CATEGORY> are replaced with your own values before you send the prompt.
- All data in labs/resources/ is synthetic - it contains no real customer, payment or employee data.
- Never paste real customer, payment or employee data into a public AI tool during this course.
- AI output varies between runs, so your wording will differ from the trainer's. Judge the output against the lab's 'Test it' step, not against the trainer's exact wording.

Topic 01 — Introduction to AI for Retail

The AI landscape · use cases across the retail value chain · hands-on with ChatGPT, Claude and Copilot · AI-generated product and marketing content · customer data privacy and responsible AI

Key concepts

- Generative AI creates new content - text, images, copy - from a prompt. An AI agent goes further: it plans and carries out a multi-step task using tools and data.
- Every stage of the retail value chain has an AI use case: sourcing, merchandising, pricing, marketing, store operations, e-commerce, service and after-sales.
- An AI assistant is only as good as the brief. Role, context, task, format and constraints turn a vague answer into copy you can actually publish.
- ChatGPT, Claude, Microsoft Copilot and Google Gemini answer the same brief differently. Comparing them on your own task is how you choose the right one.
- AI cuts content production from hours to minutes for descriptions, campaigns and visuals - but every output needs a human check before a customer sees it.
- Customer data is personal data. Anonymise before you paste, keep payment and identity data out of public AI tools, and keep a person accountable for the decision.

Lab 1 — Brief an AI Assistant Like a Retail Manager

Learning outcome: brief an AI assistant with the five-part prompt pattern and choose the right assistant for a retail task.

Goal: Three AI assistants, one retail brief. You will send a deliberately vague request, see what comes back, then rewrite it with the five-part pattern - role, context, task,

format, constraints - and run the improved brief across ChatGPT, Claude and Copilot or Gemini. You finish with a house prompt template you will reuse in every later lab.

What you'll build

A reusable five-part house prompt template, plus a scored comparison of three AI assistants on the same retail brief. (Tools: ChatGPT, Claude, Microsoft Copilot or Google Gemini, labs/resources/products.csv.)

Step-by-step

1. Open ChatGPT, Claude and either Microsoft Copilot or Google Gemini in three browser tabs and sign in to each. Open ChatGPT (chatgpt.com), Claude (claude.ai) and either Microsoft Copilot (copilot.microsoft.com) or Google Gemini (gemini.google.com) in three separate browser tabs and sign in to each. Keep all three open for the whole lab - the point of this lab is the comparison, not any one answer. Free accounts are enough.
2. Send this deliberately vague one-line brief to ChatGPT, and read the answer critically. In ChatGPT, start a new chat and send the vague brief exactly as written. Read what comes back and note three things: the length is arbitrary, the tone is generic, and it has invented details - a colour, a material or an origin story that you never supplied. This is what a one-line prompt buys you, and it is why AI output has a reputation for needing a rewrite.

Write a product description for a coffee mug.

3. Rewrite the same request with the five-part pattern and send it again in a NEW chat. Start a NEW chat - do not continue the previous one, or the vague answer will contaminate the next. Paste the five-part prompt. Each line does a specific job: Role sets the voice, Context supplies the only facts the model is allowed to use, Task states one instruction, Format fixes the shape of the answer, and Constraints rule out what you do not want. Compare this output against the first one - same model, same product, a different result.

Role: you are a senior retail copywriter for Bloom & Brew, a specialty home, gift and coffee retailer.

Context: the product is SKU BB-DRK-014, a 350ml double-walled stoneware mug, matte glaze, dishwasher safe, retail SGD 32.

Task: write the web product description.

Format: 60 words of body copy, then exactly five feature bullets of no more than 8 words each.

Constraints: British English, warm but not cute, no superlatives, no claims that are not in the context above.

4. Send that identical five-part prompt to Claude and to Copilot or Gemini, without changing a single word. Copy the same five-part prompt into Claude and into Copilot

or Gemini. Change nothing, not even the spacing: if you edit the prompt between assistants you are comparing prompts, not assistants. Line the three outputs up side by side in a document or a spreadsheet so you can read them together.

5. Score the three outputs 1-5 on brand fit, factual accuracy, format compliance and how long you would spend editing. Score each assistant 1-5 on four criteria and record the scores in a table: brand fit (does it sound like the store?), factual accuracy (did it stay inside the context you gave?), format compliance (60 words and exactly five bullets, or not?), and edit time (how many minutes to publish-ready?). Total the scores. The winner is the assistant you will reach for first in the labs after this one - and different assistants may well win for copy, for analysis and for spreadsheets.

6. Save your five-part prompt as a house template with placeholders, ready to reuse for any product. Replace the Bloom & Brew specifics with placeholders in angle brackets and save the result somewhere you will find it again - a note, a document, or a pinned chat. This template is your house prompt. Every later lab in this course starts from it, and every prompt you keep from today goes in the same place. A prompt library that you actually reuse is the single biggest time saver in this course.

Role: you are a senior retail copywriter for <YOUR STORE NAME>, a <YOUR CATEGORY> retailer.

Context: the product is <SKU>, <KEY SPECS>, retail <PRICE>.

Task: <THE ONE THING YOU WANT DONE>.

Format: <EXACT SHAPE OF THE ANSWER - length, sections, table columns>.

Constraints: <BRAND VOICE>, no claims outside the context above, say so if information is missing.

Test it

Open a fresh chat, paste your house template, and fill the placeholders with a DIFFERENT product from products.csv. The answer must come back in exactly the format you specified - the right length and the right number of bullets - with no correction needed from you. If you had to fix the shape of the answer, your Format and Constraints lines are still too loose: tighten them and run it again.

Note: Troubleshooting, a challenge and a reflection question for this lab are in labs/lab01-*/README.md. Use only the synthetic data supplied and accounts you are authorised to use. Never paste real customer, payment or employee data into a public AI tool.

Lab 2 — Generate a Product Catalogue with AI

Learning outcome: produce on-brand product descriptions, feature bullets and SEO metadata from a product data file.

Goal: Turn a row of a spreadsheet into web-ready product content. You will ground the assistant in the Bloom & Brew catalogue and brand voice guide, generate descriptions, feature bullets and SEO metadata for five SKUs in a single pass, then fact-check every claim back to the source file and fix the tone in one correction prompt rather than by hand.

What you'll build

Web-ready descriptions, feature bullets, SEO titles and meta descriptions for five SKUs, in brand voice, as a table you can paste into a product feed. (Tools: ChatGPT or Claude, labs/resources/products.csv, labs/resources/brand_voice.md.)

Step-by-step

1. Open products.csv and brand_voice.md, and choose five SKUs from at least three different categories. Open labs/resources/products.csv in a spreadsheet and labs/resources/brand_voice.md in a text editor or browser. Read the brand voice guide first - it is short, and it is the standard the output gets judged against. Pick five SKUs spread across at least three categories (for example one drinkware, one home fragrance, one coffee, one gift set, one homeware) so you can see whether the assistant holds the voice across different product types.
2. Attach both files to a new chat so the assistant writes from your catalogue instead of from guesswork. Start a new chat and attach both files (ChatGPT: the paperclip; Claude: the attachment button, or a Project with both files added). If your account cannot attach files, paste the five product rows and the full brand voice guide into the chat instead. This is grounding: the assistant now writes from your catalogue, and every claim it makes becomes checkable against a row you can point to.
3. Ask for description, bullets and SEO metadata for all five SKUs in one table, in one pass. Send the prompt with your five SKU codes filled in. Note the two things that make this a batch job rather than five separate ones: one table for all five SKUs, and a Format line precise enough that the columns come back ready to paste. The 'if a fact is missing write MISSING' constraint is the important one - it gives the model a legal way out, which is what stops it inventing a material or an origin to fill the gap.

Role: you are a senior retail copywriter for Bloom & Brew.

Context: use the attached products.csv for the facts and brand_voice.md for the tone. Nothing else.

Task: write web copy for these five SKUs: <SKU1>, <SKU2>, <SKU3>, <SKU4>, <SKU5>.

Format: one markdown table. Columns: SKU | Description (55-65 words) | Five feature bullets | SEO title (max 60 characters) | Meta description (max 155 characters).

Constraints: British English; use only facts present in products.csv; no invented materials, origins or awards; no superlatives; if a fact is missing write MISSING.

4. Fact-check every claim: read each description against the SKU's row and strike out anything the file does not support. Now do the part that cannot be delegated. Take each description and read it against that SKU's row in products.csv. Every material, capacity, origin, certification and price claim must trace back to a column in the file. Strike out anything that does not - a 'hand-thrown in Portugal' that appears nowhere in the data is exactly the kind of claim that becomes a customer complaint. Count the hallucinations you find; that count is your reason for keeping a human check in the process.

5. Correct the tone in one pass by naming what is wrong, instead of rewriting the copy yourself. Rather than rewriting the weak rows yourself, tell the assistant precisely what is wrong and what to preserve. Naming the fault ('superlatives and exclamation marks', 'drifts from brand_voice.md') and fixing the scope ('only those two descriptions', 'keep the word count and metadata unchanged') is what makes a correction prompt cheap. Asking it to 'make it better' is what makes it expensive.

Rows 2 and 4 drift from the brand voice guide: they use superlatives and exclamation marks.

Rewrite ONLY those two descriptions to match brand_voice.md. Keep the word count, the bullets and the metadata unchanged.

Return the corrected rows only, in the same table format.

6. Paste the finished table into your product sheet, and save the prompt that produced it into your prompt library. Copy the finished table into your product sheet or CMS export. Then save the prompt itself - with the placeholders back in - into the prompt library you started in Lab 1. Next month's catalogue update should be a paste and a fact-check, not a rewrite.

Test it

Pick one finished description and trace every claim in it to a column in products.csv. If a claim has no column behind it, the copy is not publishable - delete or correct it, then add the missing rule to your Constraints line so the next batch does not repeat it. A clean pass means every sentence is supported by data.

Note: Troubleshooting, a challenge and a reflection question for this lab are in labs/lab02-*/README.md. Use only the synthetic data supplied and accounts you are authorised to use. Never paste real customer, payment or employee data into a public AI tool.

Lab 3 — Campaign Copy and AI Product Visuals

Learning outcome: generate a complete promotional campaign and an AI product visual, then run a pre-publication check.

Goal: Build one seasonal promotion end to end. You will brief the campaign from the Bloom & Brew catalogue, generate email subject lines and body copy, three social

captions and an in-store poster headline, then create a product visual with an image model and iterate it one variable at a time. The lab ends with the pre-publication check every AI-assisted campaign needs.

What you'll build

A complete mini-campaign - email, three social captions, a poster headline and an AI-generated product visual - for one seasonal promotion. (Tools: ChatGPT or Claude, an image model (ChatGPT image generation, Gemini or Copilot Designer), labs/resources/products.csv.)

Step-by-step

1. Choose the promotion: pick one category from products.csv, set the offer and the dates, and name the audience. Open products.csv and pick one category with enough SKUs to promote - the coffee or home fragrance ranges work well. Decide the offer (for example 20% off the range, or a bundle price), the exact start and end dates, and who it is aimed at (existing loyalty customers, lapsed customers, gift buyers). Write these four facts down before you prompt: a campaign brief with a vague offer produces assets that contradict each other.
2. Brief the whole campaign in one prompt so every asset shares the same offer and voice. Attach products.csv and brand_voice.md, then send the campaign prompt with your four facts filled in. Asking for the whole kit in one prompt is deliberate - subject lines, body, captions and poster generated together share one offer, one voice and one call to action. Generated separately, they drift, and you spend the saved time reconciling them.

Role: you are a retail marketing lead for Bloom & Brew.

Context: promotion is <OFFER> on <CATEGORY>, running <START DATE> to <END DATE>, aimed at <AUDIENCE>. Product facts come from the attached products.csv; tone from brand_voice.md.

Task: produce the full campaign kit.

Format: (1) three email subject lines, each max 45 characters; (2) one email body of 120-150 words with a single call to action; (3) three social captions, each under 220 characters with two hashtags; (4) one in-store poster headline of no more than six words.

Constraints: state the offer and the end date in every asset; no superlatives; no claims outside products.csv; British English.

3. Push the subject lines further by asking for distinct angles rather than more variants. Three variants of the same idea are not three options. Asking for three distinct angles - urgency, benefit, curiosity - gives you something to actually choose between, and the bracketed character counts let you check the 45-character limit without counting by hand. Pick one and note why: that reasoning is what you reuse next campaign.

Rewrite the three subject lines so each takes a DIFFERENT angle: one urgency, one benefit, one curiosity.

Keep each under 45 characters and include the character count in brackets after each line.

4. Generate the product visual with an image prompt that names the subject, setting, light and framing. Move to your image model (ChatGPT image generation, Gemini, or Copilot Designer). A usable image prompt names four things: subject, setting, light and framing. The constraints matter as much: 'no text' avoids the garbled lettering image models produce, 'no logos or brand marks' avoids generating someone else's trademark, and reserving negative space at the top means your headline has somewhere to sit.

Create a photorealistic lifestyle image for a retail promotion.

Subject: a <PRODUCT> on a pale oak table, styled with a linen napkin and dried eucalyptus.

Setting: a bright Scandinavian-style cafe corner, soft morning light from the left, shallow depth of field.

Framing: 4:5 portrait, product in the lower third, clean negative space at the top for a headline.

Constraints: no text, no logos, no human faces, no brand marks on the product.

5. Iterate the image by changing ONE variable at a time - light, angle, styling or crop - never all at once. Change exactly one variable and regenerate. Then change one more. Iterating one variable at a time is what turns image generation from a slot machine into a controllable process - you learn which words move the result, and you can get back to a version you liked. Keep the prompt that produced your best image; it is a template for the next campaign, not a one-off.

Keep everything the same, change only the light: warm late-afternoon light from the right instead of morning light from the left.

6. Run the pre-publication check on the whole kit before anything goes live. Paste the full kit back into the assistant and ask for the check as a fix list. Note what this prompt does NOT ask for: a rewrite. You want the issues surfaced so a person decides what to change. Pay particular attention to the fourth item - if the visual is AI-generated and shows a product a customer will receive, check your platform's and your market's disclosure expectations before publishing, and never present a generated image as a photograph of the actual item. ``text Check this campaign kit and list every issue as a numbered fix list:

7. Any claim not supported by products.csv.
8. Any price, discount or date that is inconsistent between assets.
9. Any line that breaks brand_voice.md.

10. Anything that would need a disclosure because the image is AI-generated. Return only the issues and the fix for each - do not rewrite the campaign. ```

Test it

Read the campaign aloud as a customer would meet it: subject line, then email, then a social caption, then the poster. The offer, the discount and the end date must be identical in all four, and every product claim must trace to products.csv. One inconsistency between assets means the kit is not ready to schedule.

Note: Troubleshooting, a challenge and a reflection question for this lab are in labs/lab03-*/README.md. Use only the synthetic data supplied and accounts you are authorised to use. Never paste real customer, payment or employee data into a public AI tool.

Lab 4 — Customer Data Privacy and Responsible AI

Learning outcome: anonymise retail customer data before AI use and set the store's responsible-AI rules.

Goal: Before AI touches customer data, someone has to decide what it may see. You will classify the columns in the Bloom & Brew customer and transaction files, build an anonymised extract that is safe to paste into a public AI tool, draft a one-page AI usage policy for the store team, then red-team that policy to find what it fails to cover.

What you'll build

An anonymised data extract that is safe to use with a public AI tool, plus a one-page AI usage policy for your store team. (Tools: ChatGPT or Claude, a spreadsheet, labs/resources/customers.csv, labs/resources/transactions.csv.)

Step-by-step

1. Open customers.csv and transactions.csv and mark every column that could identify a real person. Open both files and go through the headers column by column. Mark the obvious identifiers first - full_name, email, phone. Then look for the less obvious ones: a postal code plus a join date plus a loyalty tier can identify one person in a small customer base, and customer_id links a transaction row straight back to the named record. Re-identification is usually a combination problem, not a single-column problem.
2. Ask the assistant to classify the columns - paste only the header row, never the customer data itself. Send the classification prompt. Note what it contains: headers only, and an explicit statement that no data was pasted. That is the habit worth building - you can get useful advice about a dataset without exposing the dataset. Compare the assistant's classification with the marks you made in step 1, and pay attention to any column it flagged that you did not.

Role: you are a data protection adviser to a retail business in Singapore.

Context: these are the column headers of two customer files. I have deliberately not pasted any data.

customers.csv: customer_id, full_name, email, phone, postal_code, join_date, loyalty_tier, marketing_optin

transactions.csv: order_id, order_date, customer_id, sku, qty, unit_price, channel, store

Task: classify every column as SAFE, IDENTIFYING or SENSITIVE for use with a public AI tool.

Format: a table of column, classification, one-line reason, and the action to take before any AI use.

Constraints: be conservative - if a column could re-identify someone in combination with another, say so.

3. In the spreadsheet, build the anonymised extract: delete the identifying columns, renumber the customers and keep only the month. Now do the work in the spreadsheet, not in the AI tool. Delete full_name, email and phone entirely. Replace customer_id with a plain running number (C001, C002, ...) in both files so the two still join but no longer point back to the source record. Truncate order_date and join_date to the month (2026-03 rather than 2026-03-14). Cut postal_code to its first two digits or delete it. Save the result as a new file - never overwrite the original - and use only that file for the rest of the day.

4. Draft the store's one-page AI usage policy from your own classification work. A policy that no one can follow protects nobody. Send the policy prompt and read the result as a shop-floor supervisor would: is each rule something you could check in ten seconds? The four sections matter more than the wording - what is banned outright, what needs anonymising first, what needs approval, and who to ask. Edit the draft to name real roles in your own business rather than generic ones.

Role: you are writing an internal policy for a six-outlet specialty retailer.

Context: staff use public AI assistants for product copy, customer replies and sales analysis. The column classification above is our starting point.

Task: write a one-page AI usage policy the store team will actually follow.

Format: sections for (1) what may never be pasted into an AI tool, (2) what must be anonymised first, (3) which outputs need human approval before use, (4) who to ask when unsure. Bullet points, plain English, no legal jargon.

Constraints: maximum 400 words; every rule must be checkable by a shop-floor supervisor.

5. Red-team the policy: ask the assistant to find the realistic ways a busy team would breach it. Red-teaming your own policy is faster than waiting for the breach. The scenarios that come back are the realistic ones - a rushed reply pasted with the customer's full email still in it, a screenshot of a sales report with names visible, a

supplier price list pasted into a public tool. Take the fixes that are small enough to survive contact with a busy Saturday and fold them into the policy.

You are a sceptical store manager under time pressure.

Task: list the five most likely ways this policy gets broken in a real store on a busy Saturday, and the smallest change to the policy that would prevent each one.

Format: a table of scenario, why it happens, policy fix.

Constraints: realistic retail scenarios only - no hypothetical attackers.

6. Add the human-in-the-loop rule: name who approves AI output before it reaches a customer, a price or an order. Finish by writing the rule the AI cannot write for you: the named person or role who approves AI output before it reaches a customer, a price tag or a purchase order. Responsible AI in retail is not a technology control; it is an accountable human at the point where the output becomes a commitment to a customer.

Test it

Search your anonymised extract for '@', for a phone-number pattern and for any full name from the original file. All three searches must return zero hits - that is what makes the extract safe to paste into a public AI tool. If anything is still found, your column list in step 3 was incomplete; fix it and search again.

Note: Troubleshooting, a challenge and a reflection question for this lab are in labs/lab04-*/README.md. Use only the synthetic data supplied and accounts you are authorised to use. Never paste real customer, payment or employee data into a public AI tool.

Topic 02 — Applying AI to Retail Operations and Customer Experience

AI-powered customer service · personalised recommendations and promotions · demand forecasting and inventory planning · pricing, merchandising and sales analytics · everyday AI workflows

Key concepts

- A service assistant grounded in your own FAQ, returns policy and product data answers accurately. An ungrounded one invents policy you will have to honour.
- Personalisation starts with segmentation: group customers by how recently, how often and how much they buy, then let AI write the offer for each segment.
- Demand forecasting turns sales history into a buying decision. AI is fastest at spotting trend, seasonality and the slow movers quietly tying up cash.

- Pricing and markdown calls need margin maths plus judgement. AI surfaces the candidates and the trade-offs; the merchant still makes the decision.
- Sales analytics with AI is conversational: ask in plain English, then verify the number against the source data before you act on it.
- A simple AI workflow is four parts - trigger, data, AI step, output - plus a human check. That is enough to automate a daily retail task with no code.

Lab 5 — Build a Grounded Store Service Assistant

Learning outcome: deploy an AI customer-service assistant grounded in your own policies, FAQ and product data.

Goal: An assistant that has read your returns policy answers correctly; one that has not will invent a policy you then have to honour. You will build a Bloom & Brew service assistant grounded in the store policies, FAQ and product catalogue, give it tone and escalation rules, then test it against real customer enquiries - including one your policy does not cover.

What you'll build

A store customer-service assistant grounded in your own policies and FAQ, with a tested escalation rule for anything it cannot answer. (Tools: ChatGPT Projects or Custom GPTs, or Claude Projects; labs/resources/store_policies.md, faq.md, products.csv, customer_feedback.csv.)

Step-by-step

1. Create a new Project (Claude) or Custom GPT (ChatGPT) named 'Bloom & Brew Service Assistant'. In ChatGPT, go to Explore GPTs and create a new GPT (or start a new Project); in Claude, create a new Project. Name it 'Bloom & Brew Service Assistant'. The reason to use a Project or Custom GPT rather than a plain chat is persistence: the knowledge files and the instructions stay attached to every future conversation, so the assistant does not have to be re-briefed each morning.
2. Add store_policies.md, faq.md and products.csv as the assistant's knowledge - this is what grounds it. Upload store_policies.md, faq.md and products.csv into the project knowledge. This is grounding, and it is the whole difference between an assistant that answers your returns policy and one that answers a generic returns policy it learned from the internet. Add nothing else - every extra document is another source it can quote at a customer, so keep the knowledge base small and authoritative.
3. Write the system instructions: role, tone, the grounding rule and the escalation rule. Paste the instructions into the system prompt or project instructions field. Read the lines in order: the role sets who it is, the grounding rule restricts where answers come from, the escalation line gives it a scripted way to fail safely, the 'never invent' line closes the most expensive failure mode, and the tone rules keep replies usable

without editing. The last line - quoting the policy line behind any returns answer - is what makes the assistant auditable when a customer disputes the outcome.

You are the customer service assistant for Bloom & Brew, a specialty home, gift and coffee retailer with six outlets and an online store.

Answer ONLY from the attached store policies, FAQ and product catalogue.

If the answer is not in those documents, say: 'I'll pass this to a store colleague who can help' and state what information the colleague will need.

Never invent a policy, a price, a delivery date or a stock level.

Tone: warm, direct, British English, no more than 120 words per reply, no exclamation marks.

Always end a reply about a return, refund or exchange with the exact policy line it is based on.

4. Test it on six real enquiries taken from customer_feedback.csv, one at a time. Open customer_feedback.csv and take six real enquiries from it - a returns question, a delivery question, a product-suitability question, a loyalty question, a complaint and an out-of-scope one. Send them one at a time in separate messages, as a customer would. Check each answer against store_policies.md yourself: the assistant quoting a policy line is only useful if the line it quoted is the right one.

A customer bought the ceramic pour-over set 26 days ago, has the receipt, and wants to exchange it for a different colour. What do I tell them?

5. Test the escalation path with a question your policy deliberately does not cover. Now send the question your policy does not answer. This is the most important test in the lab. A grounded assistant should say it cannot answer and hand over; an ungrounded one will confidently invent a price-match policy that your staff will then have to honour at the counter. If yours invents an answer, the grounding rule in your instructions is too weak - strengthen it and test again.

A customer is asking whether you will price-match a competitor's website. What do I tell them?

6. Tighten one weak answer by fixing the instructions, not by correcting the reply in the chat. Pick the weakest of the seven answers and resist the urge to correct it in the chat. Fix the instructions instead: add the missing rule, tighten the tone line, or add the policy document it needed. Then re-run that same enquiry in a fresh conversation. Correcting a chat fixes one reply; correcting the instructions fixes every reply from now on.

Test it

Ask the assistant a question your store policies genuinely do not cover - competitor price matching, or a return at 45 days. It must decline, use your escalation wording and state what the colleague will need. If it invents a policy instead, it is not ready to face a customer: strengthen the grounding rule and re-test.

Note: Troubleshooting, a challenge and a reflection question for this lab are in labs/lab05-*/README.md. Use only the synthetic data supplied and accounts you are authorised to use. Never paste real customer, payment or employee data into a public AI tool.

Lab 6 — Personalised Recommendations and Targeted Promotions

Learning outcome: segment customers from transaction data and build targeted offers and next-best-product recommendations.

Goal: Personalisation starts with segmentation, not with a clever message. You will group the Bloom & Brew customers by how recently, how often and how much they buy, define four segments with a targeted offer and channel for each, then build next-best-product recommendations for three individual customers and check that the maths behind each offer still leaves a margin.

What you'll build

Four customer segments with a targeted offer, channel and message for each, plus next-best-product recommendations for three named customers. (Tools: ChatGPT or Claude, a spreadsheet, labs/resources/transactions.csv, labs/resources/products.csv.)

Step-by-step

1. Use the anonymised extract from Lab 4, not the raw customer file - the same rule applies to your own store data. Attach the anonymised extract you produced in Lab 4 - the version with no names, emails or phone numbers and with customer_ref in place of customer_id. Everything in this lab works exactly the same on anonymised data, which is the point: personalisation does not require handing a public AI tool your customer list.
2. Ask for a recency, frequency and value summary per customer, and a proposed set of four segments. Send the segmentation prompt. Recency, frequency and value is the workhorse segmentation in retail because all three come straight out of a transaction file. The critical constraint is the reproducibility one: if the assistant states each segment as an explicit rule ('last order within 3 months AND 3 or more orders'), you can rebuild the segments in a spreadsheet next month without asking it again.

Role: you are a CRM analyst for Bloom & Brew.

Context: the attached file is an anonymised transaction extract. Columns: order_id, order_month, customer_ref, sku, qty, unit_price, channel, store.

Task: summarise each customer by recency (months since last order), frequency (orders in the period) and value (total spend), then propose four segments that cover every customer with no overlap.

Format: (1) a table of the four segments with their rule, customer count and share of total revenue; (2) the ten highest-value customers with their recency, frequency and value.

Constraints: state the exact rule for each segment so I can reproduce it in a spreadsheet; flag any customer the rules do not capture.

3. Sanity-check the segments in the spreadsheet before you build offers on top of them. Before you build campaigns on the segments, check them. In the spreadsheet, count how many customers fall into each segment using the stated rules and compare with the assistant's counts. Language models do arithmetic unreliably over long tables - the segmentation logic is usually sound, the counts are what drift. If the numbers do not match, trust your spreadsheet and tell the assistant the corrected counts.

4. Design the offer, channel and message for each segment in one pass. Now design the campaigns. The constraint that stops the most common personalisation mistake is 'do not discount to customers who are already buying at full price' - untargeted discounting spends margin on people who would have bought anyway. Different objectives per segment (reactivate, increase basket size, reward, win back) are what make this personalisation rather than a mailshot with four subject lines.

Task: for each of the four segments, design the next campaign.

Format: a table with columns - segment, objective, offer, channel, subject line or opening line (max 45 characters), and the one metric that tells us it worked.

Constraints: a different objective per segment - do not discount to customers who are already buying at full price; every offer must name a product category that exists in products.csv.

5. Build next-best-product recommendations for three individual customers from their own purchase history. Pick three customer references from different segments and ask for two next-best products each. The 'reason a store colleague could say out loud' column is the useful one - it turns an algorithmic recommendation into something a person can use at the counter. The constraint that the customer must not already own the product catches the most obvious failure of naive recommendation logic.

Task: for customer_ref <C0xx>, <C0yy> and <C0zz>, recommend the next TWO products each.

Format: a table of customer_ref, what they have bought, recommended SKU, and the one-sentence reason a store colleague could say out loud.

Constraints: recommend only SKUs in products.csv that the customer has not already bought; no recommendation may rely on a product attribute that is not in the file.

6. Check the margin: price the deepest offer against cost in products.csv before anyone approves it. Finally, price the deepest offer. Take the segment offer with the biggest discount, look up cost_price and retail_price for that category in products.csv, and work out the margin that survives the discount. If a 30% offer takes a 42% margin to 17%, that is a decision for a person with a budget, not something to approve because the AI suggested it. Recommendations are cheap; margin is not.

Test it

Take one next-best-product recommendation and check it against that customer's own rows in the extract: the recommended SKU must be a product they have NOT bought, and the stated reason must match what they actually purchased. If the reason cites a purchase that is not in their history, the recommendation was invented - re-run it with the customer's rows pasted directly into the prompt.

Note: Troubleshooting, a challenge and a reflection question for this lab are in labs/lab06-*/README.md. Use only the synthetic data supplied and accounts you are authorised to use. Never paste real customer, payment or employee data into a public AI tool.

Lab 7 — Demand Forecasting and Inventory Planning with AI

Learning outcome: turn sales history into a demand forecast, reorder quantities and a slow-mover list.

Goal: A forecast is only useful when it ends in a buying decision. You will take two years of Bloom & Brew monthly sales, have AI identify trend and seasonality by category, forecast the next three months for the top SKUs, then convert that into reorder points with lead time and safety stock - and check one of them by hand before anyone raises a purchase order.

What you'll build

A three-month demand forecast, reorder quantities with safety stock, and a slow-mover list ready for the buying meeting. (Tools: ChatGPT or Claude, a spreadsheet, labs/resources/sales_history_monthly.csv, labs/resources/products.csv.)

Step-by-step

1. Open sales_history_monthly.csv and check what you have: 24 months of units by SKU, category and month. Open sales_history_monthly.csv and get oriented before you prompt: which SKUs, which categories, which 24 months, and whether any SKU has gaps. A forecast built on a file you have not looked at is a guess with a table around it. Note the categories with obvious seasonality - coffee and home fragrance behave very differently through the year.
2. Ask for the pattern first - trend, seasonality and anomalies by category - before asking for any forecast. Ask for the pattern before the numbers. If the assistant describes a peak in November and December for gifting categories and a summer trough for hot drinks, it has read your data; if it describes a pattern that is not there, you have found that out before building a purchase order on top of it. The constraint about 24 months being short matters - two years gives you two observations of each season, which is enough to see a pattern and not enough to be certain of it.

Role: you are a demand planner for Bloom & Brew.

Context: the attached sales_history_monthly.csv holds 24 months of unit sales by SKU, category and month.

Task: describe the demand pattern for each category - underlying trend, seasonal peaks and troughs, and any month that looks like an anomaly rather than a pattern.

Format: a table of category, trend direction, peak months, trough months, and a one-line note on anomalies.

Constraints: base every statement on the data in the file; state clearly where 24 months is too short to be confident.

3. Forecast the next three months for the top ten SKUs by volume, with the method stated. Now the forecast. Three requirements make it usable: the method stated in one line (so you can judge it), a low and high case (so the buyer sees the range, not a false-precision single number), and the exclusion of SKUs with too little history (so new products do not get a confident forecast built on three data points). Read the stated seasonal uplift and ask yourself whether you agree with it.

Task: forecast units for the next three months for the ten highest-volume SKUs.

Format: a table of SKU, category, last three months actual, forecast for each of the next three months, and the method used in one line.

Constraints: state the seasonal uplift you applied and why; give a low and high case as well as the base case; do not forecast SKUs with fewer than six months of history - list those separately.

4. Convert the forecast into a reorder decision using lead time and safety stock. This is the step that turns analysis into a decision. Reorder point = expected demand over the lead time plus safety stock; order when stock on hand falls below it. Asking for a formula column is not a formality - it is how you find out that the assistant used 6 weeks as 1.5 months in one row and 2 months in another. Check that the lead-time demand column is consistent before you look at the order quantities.

Task: convert the base-case forecast into a reorder plan.

Context: supplier lead time is 6 weeks; we hold 2 weeks of safety stock; current stock on hand is in products.csv.

Format: a table of SKU, forecast monthly demand, lead-time demand, safety stock, reorder point, stock on hand, and order now yes/no with the quantity.

Constraints: show the arithmetic for each reorder point in a formula column so I can check it.

5. Find the cash problem at the other end: the slow movers and the overstocks. Overstock is the expensive half of inventory planning and the half that gets ignored, because nothing goes wrong visibly. Months of cover and cash at cost are the two numbers that make it visible - a total at the bottom of that table is usually the most persuasive line in a buying meeting. Match each slow mover to an action rather than listing them: markdown, bundle with a fast seller, return to supplier, or discontinue.

Task: list the SKUs where stock on hand exceeds six months of forecast demand, and the SKUs with no sales in the last three months.

Format: a table of SKU, category, stock on hand, months of cover, value at cost, and a recommended action - markdown, bundle, return to supplier or discontinue.

Constraints: use cost_price from products.csv for the value column; total the cash tied up at the bottom.

6. Recompute one reorder point by hand in the spreadsheet before the plan goes anywhere near a supplier. Take one SKU and recompute its reorder point in the spreadsheet from the raw numbers: monthly forecast divided by 4.33 to get a weekly rate, times 6 weeks of lead time, plus 2 weeks of safety stock. Compare with the assistant's figure. If they differ by more than rounding, the assumption it used was different from the one you stated - find it before the order goes out, because a wrong reorder point either runs you out of stock or ties up cash for a season.

Test it

Recompute one reorder point by hand: $(\text{monthly forecast} / 4.33) \times 6$ weeks lead time, plus 2 weeks of safety stock at the same weekly rate. The AI's number must match yours within rounding. If it does not, ask it to show the assumption it used for that row - the mismatch is always in the units, not the maths.

Note: Troubleshooting, a challenge and a reflection question for this lab are in labs/lab07-*/README.md. Use only the synthetic data supplied and accounts you are authorised to use. Never paste real customer, payment or employee data into a public AI tool.

Lab 8 — AI for Pricing, Merchandising and Sales Analytics

Learning outcome: analyse margin, model a markdown scenario and produce a KPI summary with recommended actions.

Goal: The margin question a retailer actually asks is which products to mark down, which to leave alone, and what it costs. You will build a margin picture by SKU and category, identify markdown and price-increase candidates, model what a 15% markdown does to profit, then produce a one-page KPI summary with three recommended actions - verifying the headline number yourself.

What you'll build

A margin and markdown review with a modelled price scenario, and a one-page sales KPI summary with three recommended actions. (Tools: ChatGPT or Claude, a spreadsheet, labs/resources/products.csv, labs/resources/transactions.csv.)

Step-by-step

1. Attach products.csv and the anonymised transaction extract, and ask for the margin picture by SKU and category. Attach both files and send the margin prompt. The formula is stated in the prompt on purpose - margin can be calculated on cost or on retail, and the two give different answers, so fixing the definition up front prevents an argument later. Asking it to flag any SKU where cost exceeds retail catches data errors that would otherwise propagate through every table after this one.

Role: you are a category manager for Bloom & Brew.

Context: products.csv holds cost_price, retail_price and stock_on_hand; the transaction extract holds units sold by SKU.

Task: build the margin picture.

Format: (1) a table by category of revenue, gross margin percent and gross margin value, sorted by margin value; (2) the ten SKUs contributing the most margin and the ten contributing the least.

Constraints: gross margin percent = (retail_price - cost_price) / retail_price; show the formula you used for each aggregate; flag any SKU where cost exceeds retail.

2. Ask for markdown candidates and price-increase candidates, each with the reason from the data. Markdown and price-increase candidates are the same analysis in two directions, which is why they are asked for together. The constraints encode real merchandising judgement: a slow seller with plenty of stock is a markdown candidate, a fast seller earning below its category average is a price-increase candidate, and neither call should be made on a handful of units. Read the reasons, not just the SKU list - a reason you disagree with is how you find the assumption behind it.

Task: identify (a) five markdown candidates and (b) five price-increase candidates.

Format: a table of SKU, category, current margin percent, units sold, months of cover, recommendation and the one-line reason from the data.

Constraints: a markdown candidate must have both high cover and low velocity; a price-increase candidate must have strong velocity and below-category margin; do not recommend a price change on a SKU with fewer than 20 units sold.

3. Model the money: what a 15% markdown does to margin, and how many extra units it must sell to break even. This is the step that stops a markdown being approved on gut feel. The break-even uplift is the number that matters: if a 15% markdown needs 40% more units just to stand still, the question is whether that is realistic for this product. The constraint about elasticity is deliberate - the model has no data on how your customers respond to price, so any elasticity it volunteers is fabricated, and asking it not to invent one keeps the output honest.

Task: model a 15% markdown on the five markdown candidates.

Format: a table of SKU, current price, marked-down price, current margin percent, new margin percent, margin value lost per unit, and the extra units needed to hold total margin flat.

Constraints: state the break-even uplift as a percentage of current units; show the arithmetic; do not assume any demand elasticity you have not been given.

4. Write the category review narrative a merchant would actually read. Numbers do not persuade on their own. The category review turns the tables into something a merchant reads in two minutes: what is working, what is not, the opportunity and the risk of inaction. The 'every claim must cite a number' constraint is what keeps it from producing plausible retail advice that would be equally true of any store in any year.

Task: write the category review for the two largest categories by revenue.

Format: for each category - what is working, what is not, the single biggest opportunity, and the risk if we do nothing. Maximum 120 words per category.

Constraints: every claim must cite a number from the analysis above; no generic retail advice.

5. Produce the one-page KPI summary for management, ending in three actions with an owner and a date. The KPI summary is the artifact that leaves the room. Three parts make it actionable: headline KPIs with direction, three actions each with an impact, an owner role and a date, and one risk flagged. Actions without an owner and a date are observations. Edit the draft so the owner roles match real roles in your business.

Task: write a one-page sales KPI summary for the monthly management meeting.

Format: (1) five headline KPIs with the number and the direction; (2) three recommended actions, each with the expected impact, a suggested owner role and a by-when; (3) one risk to flag.

Constraints: maximum 350 words; plain English; no recommendation without a number behind it.

6. Verify the headline number yourself before the summary is circulated. Before circulating anything, verify the headline. Pick the single number the summary leads with - total gross margin, or the margin percentage for the biggest category - and recompute it in the spreadsheet from the source columns. Circulating an AI-generated number you have not checked is how a management meeting ends up making a decision on a hallucinated figure.

Test it

Verify the gross margin the AI reports for one SKU: $(\text{retail_price} - \text{cost_price}) / \text{retail_price}$ from products.csv, worked out in the spreadsheet. It must match to one decimal place. Then recompute the total margin value for the largest category the same way. If either differs, the aggregation is wrong and every table built on it needs re-running from the corrected figures.

Note: Troubleshooting, a challenge and a reflection question for this lab are in [labs/lab08-*/README.md](#). Use only the synthetic data supplied and accounts you are authorised to use. Never paste real customer, payment or employee data into a public AI tool.

Lab 9 — Build a Simple AI Workflow for a Daily Retail Task

Learning outcome: build and document a repeatable no-code AI workflow with a human check for a daily retail task.

Goal: The last step is turning today's prompting into something that runs tomorrow without you. You will build a no-code daily workflow in a spreadsheet - trigger, data, AI step, output, human check - that produces the Bloom & Brew daily sales summary and drafts replies to overnight customer reviews, then document it as a one-page SOP and prove it repeats on a second day of data.

What you'll build

A working no-code daily-summary workflow - trigger, data, AI step, output, human check - documented as a one-page SOP your team can run. (Tools: Google Sheets or Excel, ChatGPT or Claude, [labs/resources/transactions.csv](#), [labs/resources/customer_feedback.csv](#).)

Step-by-step

1. Define the workflow on paper first: trigger, data in, AI step, output, human check - five boxes, one line each. Before touching a tool, write the five boxes down. Trigger: 9am each day. Data in: yesterday's transactions and overnight feedback. AI step: the fixed summary prompt. Output: the daily summary and draft replies. Human check: the duty manager approves before anything is sent. Every no-code AI workflow is these five boxes - a workflow that goes wrong is almost always missing the fifth.
2. Set up the sheet: one tab for the day's transactions, one for overnight feedback, one for the output. Create a spreadsheet with three tabs: Data (paste the day's transaction rows), Feedback (paste overnight reviews from [customer_feedback.csv](#)), and Output (where the finished summary is pasted back). Filter [transactions.csv](#) to a single day for the Data tab. The sheet is deliberately the simplest possible plumbing - the point of this lab is the repeatable pattern, not the tooling, and the same five boxes carry over to any automation platform you adopt later.
3. Write the AI step as a fixed, reusable prompt with the day's data as the only thing that changes. Write the AI step as a fixed prompt. The only thing that changes between runs is the pasted data - if the prompt itself changes daily, you have not built a workflow, you have just done the task again. Two constraints do the heavy lifting: 'NOT AVAILABLE' for anything not calculable stops invented numbers, and the ban on promising refunds keeps a draft reply from committing the store to something before a person has seen it.

Role: you are the duty manager writing the Bloom & Brew daily trading summary.

Context: below are yesterday's transaction rows and yesterday's customer feedback. Nothing else.

Task: write the daily summary.

Format: (1) five bullets - revenue, units, best category, worst category, one thing that stands out; (2) any stock or service issue that needs attention today; (3) a draft reply to each piece of feedback rated 3 stars or below, maximum 60 words each, in our house tone.

Constraints: use only the rows below; if a number cannot be calculated from them, write NOT AVAILABLE; never promise a refund or a discount in a draft reply - offer to have a colleague make contact.

- 4.** Run the workflow end to end on one day of data and time how long it takes. Run it: paste the day's rows and the feedback into the prompt, send it, paste the result into the Output tab. Time it. Compare that with how long the same summary takes by hand today. That number is what you take back to your manager, and it is also the honest test of whether the workflow is worth keeping.
- 5.** Add the human check that has to happen before anything is sent, and name who does it. Write the human check into the sheet as a literal step with a name against it: who reads the summary, who approves each draft reply, and what they check (numbers against the Data tab, replies against store policy). An approval step that is not written down is not a control - it is a hope.
- 6.** Document the workflow as a one-page SOP so someone else can run it tomorrow. Finally, have the assistant write the SOP, then read it as if you had never seen the workflow. Can a duty manager follow it on their first morning? Does it include the exact prompt to paste and what to do when the output looks wrong? Save the SOP with the sheet - the workflow that survives is the one someone else can run without you.

Task: turn the workflow I have just built into a one-page standard operating procedure.

Format: purpose, when it runs, who runs it, the numbered steps with the exact prompt to paste, what to check before sending, and what to do when the output looks wrong.

Constraints: written for a duty manager who has never used AI before; maximum one page; no jargon.

Test it

Run the whole workflow again on a SECOND day of data without changing a word of the prompt. If the output comes back in the same format, with the numbers matching what the Data tab shows and no invented figures, the workflow is repeatable and ready for your team. If you had to adjust the prompt to make day two work, it is still a manual task - fix the prompt, not the day.

Note: Troubleshooting, a challenge and a reflection question for this lab are in [labs/lab09-*/README.md](#). Use only the synthetic data supplied and accounts you are authorised to use. Never paste real customer, payment or employee data into a public AI tool.

Wrap-Up - Putting AI to Work in Your Store

You have now taken AI through a full retail day: briefing an assistant, producing catalogue and campaign content, protecting customer data, grounding a service assistant, personalising offers, forecasting demand, reviewing pricing and automating a daily task. The pattern is the same every time - brief it properly, ground it in your own data, verify the output, then keep the prompt.

The five-part prompt pattern, once more

- Role: who the AI is being asked to be (a retail copywriter, a category manager, a demand planner).
- Context: the store, the customer, the season, the constraint - plus the data file if there is one.
- Task: the single thing you want done, stated as an instruction, not a question.
- Format: the exact shape of the answer - a table, 60 words, five bullets, a subject line.
- Constraints: brand voice, what it must not claim, what it must do if it does not know.

What must always stay human

- Any claim about a product that a customer could rely on.
- Any price, markdown or purchase-order quantity.
- Any reply that commits the store to a refund, exchange or goodwill gesture.
- Any decision that uses customer data - and the decision about what data is used at all.

Where the time savings actually come from

- Reuse: a saved prompt that works is worth more than a one-off good answer.
- Grounding: attaching your own policy, FAQ and product data removes most of the correction work.
- Format discipline: asking for a table you can paste beats asking for prose you must retype.
- Batching: doing five SKUs in one prompt costs barely more time than doing one.

Next Steps

- First pass: complete every lab yourself, following the steps in this guide and the lab READMEs.
- Second pass: re-run the labs on your own product, sales and customer data instead of the Bloom & Brew files.
- Assemble your prompt library - each working prompt saved with the output format it produced.
- Agree your store's AI usage policy (Lab 4) with your team before AI touches customer data or pricing.
- Pick one daily task and put the Lab 9 workflow into live use, with the human check kept in place.

Glossary

- **Generative AI** — AI that produces new content - text, images, audio, code - in response to a prompt, rather than only classifying or predicting.
- **Large Language Model (LLM)** — The model behind an AI assistant. It predicts text, which is why it is fluent but can also be confidently wrong.
- **AI Assistant** — A chat product built on an LLM - ChatGPT, Claude, Microsoft Copilot, Google Gemini - that you brief in plain English.
- **AI Agent** — An AI that plans and carries out a multi-step task using tools, files or systems, rather than only answering a single question.
- **Prompt** — The instruction you give an AI assistant. The five-part pattern used in this course is role, context, task, format, constraints.
- **Grounding** — Supplying the AI with your own documents or data so its answers come from your store's facts instead of general internet knowledge.
- **Hallucination** — A fluent, confident output that is factually wrong - an invented product feature, policy or number. The reason every output is checked.
- **Human in the Loop** — A named person who reviews and approves AI output before it reaches a customer, a price tag or a purchase order.
- **Personal Data** — Any data that identifies a customer - name, email, phone, address, payment details. Removed before data is used with a public AI tool.
- **Anonymisation** — Removing or replacing identifying fields so a data extract can no longer be traced to an individual customer.
- **Segmentation** — Grouping customers by behaviour - typically how recently, how often and how much they buy - so offers can be targeted.
- **Next-Best-Product** — The product a specific customer is most likely to buy next, inferred from their purchase history and that of similar customers.
- **Demand Forecast** — A projection of future unit sales for a product or category, used to decide what and when to reorder.
- **Reorder Point** — The stock level that triggers a new order: expected demand over the supplier lead time, plus safety stock.

- **Safety Stock** — Extra stock held to absorb demand spikes and late deliveries without going out of stock.
- **Gross Margin** — $(\text{Retail price} - \text{cost price}) / \text{retail price}$, as a percentage. The core profitability measure in the pricing lab.
- **Markdown** — A permanent or promotional price reduction used to clear stock, at the cost of margin.
- **Slow Mover** — A product selling far below the assortment average, tying up cash and shelf space.
- **AI Workflow** — A repeatable sequence - trigger, data, AI step, output, human check - that automates a recurring task without code.
- **Brand Voice** — The documented tone and language rules that keep every piece of customer-facing content sounding like the same store.